

实验三 序列检测器设计报告

姓名：怀迪 学号：PB23061112

一、实验内容简述

本次实验要求我们编写 Verilog 代码实现检测我们所需要的序列的电路设计，并在 Quartus 软件中进行仿真和 FPGA 验证。

需要实现的具体功能：首先需要两个序列产生器用来产生两组不同的序列信号，然后需要一个选择器用来选择其中一组序列信号进行检测，最后需要一个序列检测器用来检测这两组序列信号中是否包含我们所需要的序列。

二、设计分析

1. 实验题目分析

我们需要设计一套包含“序列生成 - 路径选择 - 序列检测”的完整系统，实现对特定串行序列“111010011”的精准识别：

- ①系统接收两路可选输入序列（一路含目标序列，一路不含）；
- ②通过选择器切换输入路径；
- ③序列检测器实时监测输入序列，检测到目标序列时输出“1”，未检测到则输出“0”。

2. 编程思路

我把代码分成了五个模块：实现具体功能的顶层模块，分频模块，序列产生模块，序列选择模块，检测序列模块。

2.1 顶层模块

顶层模块只需调用其他模块即可实现具体功能的实现。

首先调用分频模块对 clk 信号进行分频,得到一个较低频率的时钟信号 clk_divided, 本次实验我选择将时钟分频成 2Hz, 以便于人眼观察。

然后调用产生两个序列的模块, 与实验指导 pdf 不同的是, 我选择循环计数的方式来产生序列信号, 使用一个 case 语句根据所记数字选择对应数值进行输出。

接着调用选择器根据 0/1 输入选择对应的序列进行输出。

最后调用序列检测模块把相应序列输入到序列检测器中进行检测。

2.2 分频模块

分频模块主要实现对输入的 clk 信号进行分频操作。

首先输入未分频的时钟信号, 使用一个计数器对输入的 clk 信号进行计数, 当计数器达到设定的分频值时, 翻转输出的分频信号, 并将计数器清零。

此时得到的输出信号为分频后的时钟信号 clk_divided。

2.3 序列产生模块

首先选择器根据输入的时钟信号进行计数, 每次计数后都输出一个 1/0 信号。

循环往复实现序列的循环输出。

2.4 序列选择模块

检测选择的输入, 运用 case 语句根据选择信号选择对应的序列进行输出。

2.5 序列检测模块

使用状态机的方式实现对序列的检测。

设计一个状态机, 包含多个状态, 每个状态对应序列中的一个数字。每个时钟周期根据输入信号和当前状态进行状态转移。当状态机达到最终状态时, 输出检测到目标序列的信号使 LED 灯亮起。

同时, 我还编写了一个数码管的代码显示当前状态, 方便调试和观察。

三、VHDL 源代码

本代码实现了分频时钟作为输入, 手动选择输出序列, 选择过后使能模块, 使能过后实验箱根据序列 LED 灯周期性的亮灭或者一直熄灭。

3.1 顶层模块

```
3 module FPGA_EXP3_hd (
4     input clk,
5     input ena,
6     input choose,
7     output signal, // 选择一个LED灯进行显示
8     output [6:0] LED_data, // 选择一个数码管显示状态
9     output LED_choose
10 );
11 wire seq1;
12 wire seq2;
13 wire seq_in;
14 wire clk_divided;
15
16 FPGA_EXP3_div div // 分频时钟信号为2Hz
17     .clk(clk),
18     .clkout(clk_divided)
19 );
20
21 seq_generate1 generate1 // 产生序列1
22     .clk(clk_divided),
23     .ena(ena),
24     .seq1(seq1)
25 );
26
27 seq_generate2 generate2 // 产生序列1
28     .clk(clk_divided),
29     .ena(ena),
30     .seq2(seq2)
31 );
32
33 seq_choose choose2_1 // 序列信号二选一
34     .choose(choose),
35     .seq1(seq1),
36     .seq2(seq2),
37     .seq_out(seq_in)
38 );
39
40 seq_detector detect(
41     .clk(clk_divided),
42     .ena(ena),
43     .seq_in(seq_in),
44     .seq_out(signal),
45     .LED_data(LED_data), // 数码管显示序列状态
46     .LED_choose(LED_choose)
47 );
48 endmodule
```

3.2 分频模块

```
1 `timescale 10ns/1ns
2
3 module FPGA_EXP3_div(
4     input clk,
5     output reg clkout
6 );
7     // 分频 (通过计数器实现 50MHz-2Hz)
8     reg [23:0] cnt; // 计数器
9     always @(posedge clk)
10         begin
11             if (cnt >= 12499999)
12                 begin // 50MHz/2/12500000 = 2Hz
13                     cnt <= 1'b0;
14                     clkout <= ~clkout; // 翻转输出, 生成 2Hz 方波
15                 end
16             else
17                 begin
18                     cnt <= cnt + 1'b1;
19                 end
20             end
21 endmodule
```

3.3 序列产生模块

两个序列产生模块代码逻辑基本一致，只是输出的序列不同。所以我只展示其中一个模块的代码。

```
3 module seq_generatel(//产生序列1
4     input clk,
5     input ena,
6     output reg seq1
7 );
8     reg [3:0] num; //用来循环计数输出序列111010011
9     always @(posedge clk) //循环产生序列
10 begin
11     if(ena)
12     begin
13         if(num == 4'b1000)
14         begin
15             num <= 4'b0000;
16         end
17         else if(num >= 4'b0000 && num <= 4'b0111)
18         begin
19             num <= num + 4'b0001;
20         end
21         else
22         begin
23             num <= 4'b0000;
24         end
25     end
26     case (num)
27         4'b0000: begin seq1 <= 1'b1; end
28         4'b0001: begin seq1 <= 1'b1; end
29         4'b0010: begin seq1 <= 1'b1; end
30         4'b0011: begin seq1 <= 1'b0; end
31         4'b0100: begin seq1 <= 1'b1; end
32         4'b0101: begin seq1 <= 1'b0; end
33         4'b0110: begin seq1 <= 1'b0; end
34         4'b0111: begin seq1 <= 1'b1; end
35         4'b1000: begin seq1 <= 1'b1; end
36         default: begin seq1 <= 1'bx; num <= 4'b1000; end
37     endcase
38 end
39 else if(!ena)
40 begin
41     seq1 <= 1'bx;
42     num <= 4'b1000;
43 end
44 end
45 endmodule
```

3.4 序列选择模块

```
1 `timescale 10ns/1ns
2
3 module seq_choose(//序列信号二选一
4     input choose,
5     input seq1,
6     input seq2,
7     output reg seq_out
8 );
9     always @(*)
10 begin
11     case (choose)
12         1'b0: seq_out <= seq1;
13         1'b1: seq_out <= seq2;
14         default: seq_out <= 1'bx;
15     endcase
16 end
17 endmodule
```

2.5 序列检测模块

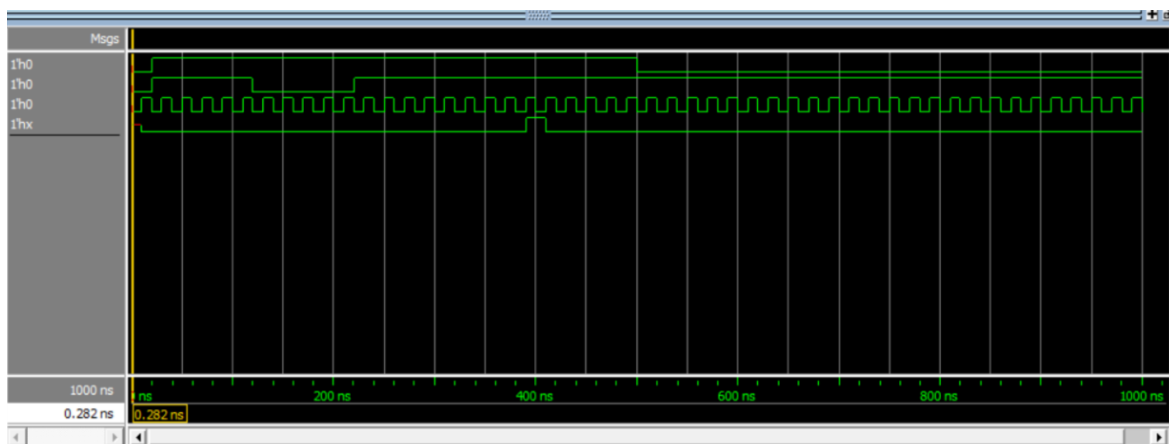
```
3 module seq_detector(  
4     input clk,  
5     input ena,  
6     input seq_in,  
7     output reg seq_out,  
8     output reg [6:0] LED_data, //数码管显示序列状态  
9     output reg LED_choose  
10 );  
11     reg [3:0] st;  
12     parameter s0=0,s1=1,s2=2,s3=3,s4=4,s5=5,s6=6,s7=7,s8=8,s9=9;  
13     always @(posedge clk) begin  
14         if(!ena)  
15             begin  
16                 seq_out <= 0;  
17                 st<=s0;  
18                 LED_choose <= 1'b1;  
19                 LED_data <= 7'b0001000; //A  
20             end  
21         else if(ena)  
22             begin  
23                 case (st)  
24                     s0: begin  
25                         seq_out <= 0;  
26                         LED_data <= 7'b00000001; //0  
27                         LED_choose <= 1'b1;  
28                         if (seq_in==1'b0)  
29                             st<=s0;  
30                         else if(seq_in==1'b1)  
31                             st<=s1;  
32                         else  
33                             st<=s0;  
34                     end  
35                     s1: begin  
36                         seq_out <= 0;  
37                         LED_choose <= 1'b1;  
38                         LED_data <= 7'b1001111; //1  
39                         if (seq_in==1'b0)  
40                             st<=s0;  
41                         else if(seq_in==1'b1)  
42                             st<=s2;  
43                         else  
44                             st<=s0;  
45                     end  
46                     s2: begin  
47                         seq_out <= 0;  
48                         LED_choose <= 1'b1;  
49                         LED_data <= 7'b0010010; //2  
50                         if (seq_in==1'b0)  
51                             st<=s0;  
52                         else if(seq_in==1'b1)  
53                             st<=s3;  
54                         else  
55                             st<=s0;  
56                     end  
57                     s3: begin  
58                         seq_out <= 0;  
59                         LED_choose <= 1'b1;  
60                         LED_data <= 7'b0000110; //3  
61                         if (seq_in==1'b0)  
62                             st<=s4;  
63                         else if(seq_in==1'b1)  
64                             st<=s3;  
65                         else  
66                             st<=s0;  
67                     end  
68                     s4: begin  
69                         seq_out <= 0;  
70                         LED_choose <= 1'b1;  
71                         LED_data <= 7'b1001100; //4  
72                         if (seq_in==1'b0)  
73                             st<=s0;  
74                         else if(seq_in==1'b1)
```

```

75         st<=s5;
76     else
77         st<=s0;
78     end
79 s5: begin
80     seq_out <= 0;
81     LED_choose <= 1'b1;
82     LED_data <= 7'b0100100; //5
83     if (seq_in==1'b0)
84         st<=s6;
85     else if(seq_in==1'b1)
86         st<=s2;
87     else
88         st<=s0;
89     end
90 s6: begin
91     seq_out <= 0;
92     LED_choose <= 1'b1;
93     LED_data <= 7'b0100000; //6
94     if (seq_in==1'b0)
95         st<=s7;
96     else if(seq_in==1'b1)
97         st<=s1;
98     else
99         st<=s0;
100    end
101 s7: begin
102     seq_out <= 0;
103     LED_choose <= 1'b1;
104     LED_data <= 7'b0001111; //7
105     if (seq_in==1'b0)
106         st<=s0;
107     else if(seq_in==1'b1)
108         st<=s8;
109     else
110         st<=s0;
111    end
112 s8: begin
113     seq_out <= 0;
114     LED_choose <= 1'b1;
115     LED_data <= 7'b0000000; //8
116     if (seq_in==1'b0)
117         st<=s0;
118     else if(seq_in==1'b1)
119         st<=s9;
120     else
121         st<=s0;
122    end
123 s9: begin
124     seq_out <= 1;
125     LED_choose <= 1'b1;
126     LED_data <= 7'b0000100; //9
127     if (seq_in==1'b0)
128         st<=s0;
129     else if(seq_in==1'b1)
130         st<=s1;
131     else
132         st<=s0;
133    end
134 default:begin
135     seq_out <= 1'bx;
136     LED_choose <= 1'b1;
137     LED_data <= 7'b0111000; //F
138     st<=s0;
139     end
140 endcase
141 end
142 end
143
144 endmodule

```

四、仿真结果记录



仿真结果如上图所示，首先 choose 信号（第一行）置一，选择所需序列，然后使能信号（第二行）置一，模块开始工作，可以看到初始 signal（第四行）输出一直为 0，而后使能信号置零，模块停止工作，signal 输出保持不变。经过一段时间后，使能信号再次置一，模块继续工作，signal 在经过一段时间后变为 1，表明检测到了所需序列。

随后 choose 信号置零，选择错误序列，然后使能信号置一，模块开始工作，可以看到 signal 输出一直为 0，表明没有检测到所需序列。

仿真结果与预期相符，代码逻辑正确。

五、FPGA 验证结果记录

我选择 DIP0 作为使能信号输入，DIP1 作为序列选择输入信号 (为 0 输入所需序列，为 1 输入错误序列)，一个 LED 灯用来显示是否检测到所需序列，一个数码管用来显示当前序列检测器所处状态。

第一次我将 DIP1 置 0，即选择所需序列，然后将 DIP0 置 1，使能整个模块，可以看到数码管循环显示 0-9，并且在数码管显示 9 的同时 LED 灯会亮起，表明检测到了所需序列，与理论相符。

随后我将 DIP1 置 1，输出另一个错误的序列，然后将 DIP0 置 1，使能整个模块，可以看到数码管显示 0-4 后又显示回了 0，这表明检测到的序列不是所需序列，LED 灯也始终没有亮起，与理论相符。

六、实验总结

有了实验二的经验，本次实验相对来说轻松了许多，比如说分频模块可以直接借鉴 ()。

由于老师和助教都推荐把代码分模块编写，本次实验我选择把不同的模块都单独放到不同的文件里，然后在顶层模块里直接调用相关模块。可以看出整个代码确实清爽了不少，哪个模块有问题也可以单独修改，便利了不少。

本次实验过程中比较顺利，并未遇到什么问题，希望我后面的代码逻辑能越来越清晰简洁。